SWE-bench on AgentCore

Image sizes, the arm64 and 2 GB limits, and three sandbox demos for agentic RL: AgentCore, EKS, ECS

Kaige Yang
Senior Applied Scientist, AWS

Data as of September 28, 2026

Agenda

01 Background

What an SWE-bench task needs from a sandbox

02 Survey

All 500 Verified images vs the AgentCore limits, probed on AWS

03 Challenges and solutions

arm64, the 2 GB cap, and a two-lane design

04 What we built

AgentCore (first run, arm64), EKS + gVisor, ECS: limits of each, compared

Appendix (from slide 28): per-repo detail and references.

© 2026, Amazon Web Services, Inc. or its affiliates.

Most SWE-bench rows qualify for AgentCore microVMs on architecture and size; an m5 lane covers the tail

Goal	RL-train a coding agent (Qwen3-4B, slime GRPO) on **SWE-bench**, each episode in its own isolated AgentCore sandbox.
Background	SWE-bench needs **one image per dataset row** (repo + commit + environment); the official images are amd64, p50 1.08 GB compressed.
Limits, probed	The 2 GB cap counts the **compressed** size and is enforced when the Runtime is created (**2048 MiB**: 2,042 accepted, 2,062 rejected; 3 GB on disk / 64 MB compressed accepted). Cold open 22–30 s at 1.5–2 GiB; ≈ 10 pre-warmed per new Runtime.
arm64	microVMs are arm64-only, but **prebuilt arm64 images exist for 420 / 500 rows**, all ≤ 2 GB (Epoch AI on GHCR; 281 also official) (slide 10).
Solutions	Re-wrap the existing arm64 images, rebuild the rest per (repo, version) environment, slim what's over 2 GB, and route the remaining rows to an **m5 Instance lane**.
Not yet shown	No real SWE-bench image has run on a microVM yet: the 420 are *eligible*, not validated. 139 of them exist only as Epoch builds, assumed to keep the official `/testbed` layout.
Built on AgentCore	`demo_cpu_swebench`, 20 tasks. First run (m5 lane): 24 episodes, **0 infra errors**, the ≈ 90 s boot hidden; 0 solved. Now on **arm64 microVMs**: local oracle check passes, AWS run pending (slide 20).
EKS and ECS twins (not deployed)	Same rows, grader and amd64 images: one **gVisor pod** (`_eks`) or one **ECS task** (`_ecs`) per episode on shared m5 nodes; no image limits, weaker boundaries (slide 26).

Source: `code/demos/demo_cpu_swebench/` (`survey_image_sizes.py`, `probe_image_cap.py`, smoke log 2026-09-28); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every SWE-bench row is its own repo, commit and environment, so one image can't serve the dataset

Table 1 **The task**

Input	A real GitHub issue + the repo checked out at `base_commit`, dependencies installed
Agent's job	Explore, edit, run code: a multi-turn tool loop (`bash` / `read_file` / `write_file`)
Reward (RLVR)	Hidden `test_patch` applied **after** the agent stops; solved iff every FAIL_TO_PASS and PASS_TO_PASS test passes. No LLM judge
Verified	**500** human-validated instances from **12** Python repos
Training scale	Full test set 2,294; SWE-Gym **2,438**; SWE-smith **59,136**; same per-instance format

- **A sandbox problem:** each row has a different repo, commit and dependency set. The official harness ships one prebuilt image per instance, `swebench/sweb.eval.x86_64.<id>`.
- **RL adds scale:** NS GRPO siblings per task per step, each in a **pristine** copy of the environment, thousands of episodes per run.

In plain English: every exercise comes with its own computer, and each attempt needs a clean copy of it.

Source: `princeton-nlp/SWE-bench_Verified`, `SWE-Gym/SWE-Gym` (train 2,438), `SWE-bench/SWE-smith` (train 59,136) on Hugging Face; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Each episode needs the row's exact environment, a fresh machine and a route back to the trainer

Table 2 What the sandbox must provide per episode

Need	Why	AgentCore mechanism
The row's exact environment	Tests only mean something in the environment SWE-bench validated	A custom image per row (or per environment) → Runtime
Fresh state every episode	The agent edits files and installs packages; GRPO siblings must not see each other	One brand-new `runtimeSessionId` = a fresh microVM or instance
Minutes of tool calls + a test run	Our grader allows each test run up to 600 s	15-minute request timeout; heartbeat past the idle timeout
A route back to the policy	The agent loop runs **in** the session and calls the trainer	VPC mode, SG→SG rule to the adapter ports
Hidden tests	Stops reward hacking	`test_patch` lives only on the trainer, injected at grading

Takeaway: AgentCore already has every mechanism; the open questions are the image (arm64, size), not the session model.

Source: AgentCore developer guide [D1] and quotas [Q1]; project design in `demo_cpu_swebench`; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

500 Verified rows collapse to 80 environments, and django alone is 46 % of them

Table 3 SWE-bench Verified by repo (500 instances)

Repo	Inst.	Vers.	Repo	Inst.	Vers.
django	**231**	9	astropy	22	6
sympy	75	12	xarray	22	4
sphinx	44	15	pytest	19	10
matplotlib	34	6	pylint	10	5
scikit-learn	32	4	requests	8	7
seaborn	2	1	flask	1	1
Total				**500**	**80**

Source: `princeton-nlp/SWE-bench_Verified` (test split), grouped by `repo` and `version`; checked 2026-09-28.

- **Only 80 distinct (repo, version) environments.** Instances of one environment differ only in the commit checked out.
- That makes **one image per environment** possible: 500 → 80 images and Runtimes (slide 15).

© 2026, Amazon Web Services, Inc. or its affiliates.

Two microVM limits matter: arm64 only, and a 2 GB image cap that can’t be raised

Table 4 **The AgentCore limits that matter**

Limit	Value	Ref
microVM CPU architecture	**arm64 only**	[D1]
Max image size (L-0A9E32B3)	**2048 MiB compressed, not adjustable**	[Q2][P1]
Runtimes per account	1000	[Q1]
Active / new sessions	5000 / 25 per s	[Q2]
Capacity-provider Instances (m5, g6)	amd64, no 2 GB cap	[D2]

- The quota text says “maximum container size” without saying compressed or on disk, so **we probed it on AWS** (slide 8).
- Instances have no cap and run amd64, but **1 session = 1 instance**: a whole m5 per episode.

Source: `aws service-quotas get-service-quota --service-code bedrock-agentcore` (us-west-2) [Q2]; AgentCore docs [Q1][D1]; project measurement [D2]; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

The cap is 2048 MiB of compressed image, enforced when the Runtime is created

Table 5 **Cap probe: one 61 MiB base image + a filler layer, arm64, VPC mode**

Filler	Compressed	Create	Sessions
3 GiB zeros (3.1 GiB on disk)	64 MiB	**READY**	5/5
1,000 MiB random	1,061 MiB	READY	5/5
1,500 MiB random	1,562 MiB	READY	5/5
1,890 MiB random	1,952 MiB	READY	13/13
1,940 MiB random	2,002 MiB	READY	13/13
1,980 MiB random	**2,042 MiB**	**READY**	13/13
2,000 MiB random	**2,062 MiB**	**rejected**	—
2,300 MiB random	2,362 MiB	**rejected**	—

Rejection: `ServiceQuotaExceededException: maxImageSizeMb limit exceeded`, returned synchronously by `CreateAgentRuntime`. MiB = 2^{20} bytes of ECR `imageSizeInBytes`.

- **Compressed, not on disk:** 3.1 GiB on disk passes when it compresses to 64 MiB.
- **Binary, not decimal:** 2,042 MiB (2.14 GB decimal) passes, 2,062 MiB fails, so the limit sits in (2,042, 2,062] MiB, consistent with 2048 MiB.
- **Budget:** our wrapper adds 54 MiB, so a source image must be $\leq \approx$ 1,990 MiB. The largest arm64 image (1,772 MiB) has 220 MiB spare.
- **Fails fast** at create time, so a deploy script sees it at once.

Source: `probe_image_cap.py` → `swebench/capprobe.csv`, `capprobe_edge.csv`; us-west-2, two runs 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Near the cap a cold open takes 28–30 s, but about 10 opens per new Runtime find a microVM already running

Table 6 Session open times from the cap probes (54 opens)

Image	Opens	Cold	Warm open	Cold open (pull+boot)
64 MiB	5	0	0.43–0.56 s	—
1,061 MiB	5	0	0.44–0.59 s	—
1,562 MiB	5	1	0.55–0.67 s	23.5 s (22.1)
1,952 MiB	13	3	0.63–0.89 s	27.8–27.9 s (27.1–27.5)
2,002 MiB	13	3	0.48–0.95 s	29.9–30.0 s (28.1–28.2)
2,042 MiB	13	3	0.55–1.01 s	29.9–30.0 s (28.2–28.3)

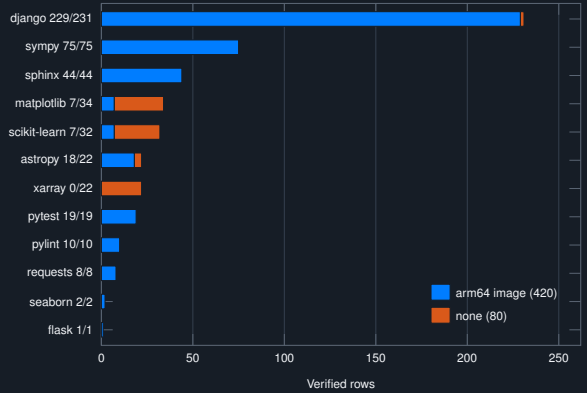
- **Warm:** the server had started before the open was sent, so AgentCore *appears* to pre-warm microVMs for a new Runtime (inferred, not documented).
- **Depth ≈ 10:** 1 open then a burst of 12 gave exactly 10 warm + 3 cold on all three Runtimes. Refill under sustained load is not measured.
- **Cold:** 22 s at 1.5 GiB, 28–30 s near 2 GiB (n = 10), still **≈ 3× faster** than the ≈ 90 s m5 boot (slide 19).

Source: `probe_image_cap.py`: 1 open, then a burst of 4 (first run) or 12 (edge run) per Runtime; warm = server started before the open was sent; us-west-2, 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

arm64 images already exist for 420 of 500 rows, all under 2 GB; 80 rows have none

Figure 1 Verified rows per repo with a prebuilt arm64 image (any registry)



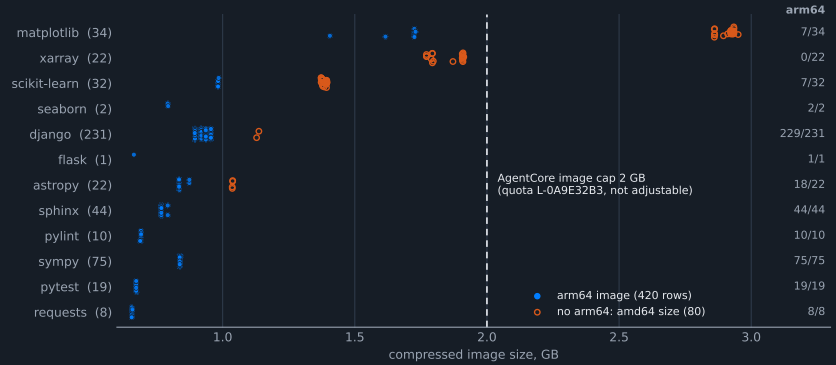
- **Epoch AI (GHCR):** 420 / 500 arm64, p50 0.89 GB, max 1.73 GB.
- **Official (Docker Hub):** 281 / 500, all also on Epoch; p50 1.00 GB, max 1.94 GB.
- **All 420 fit the cap.** Median size vs amd64: official arm64 0.92×, smallest arm64 per row 0.83×.
- **Assumption:** the 139 Epoch-only rows keep the official `/testbed` + conda env `testbed` layout (not yet checked); every image still needs the oracle check (slide 14).
- **The 80 without:** matplotlib 27, scikit-learn 25, xarray 22, astropy 4, django 2 → rebuild or the m5 lane.

Source: `survey_image_sizes.py` (Docker Hub + GHCR [S1][S2]); compressed sizes, architecture read from each image config; checked 2026-09-28. Per repo: slide 29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every existing arm64 image fits the cap; of the 80 rows without one, 27 are over 2 GB even on amd64

Figure 2 Per Verified row: smallest prebuilt arm64 image (filled), or the amd64 size where none exists (hollow)



Source: Docker Hub `swebench/sweb.eval.{arm64,x86_64}.*`, GHCR `epoch-research/swe-bench.eval.arm64.*`; compressed sizes, all 500 rows (`survey_image_sizes.py`); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

The official amd64 images: median 1.08 GB compressed, about 3× that on disk

Table 7 **Official SWE-bench Verified images (amd64, compressed)**

min / p50 / p90 / max	0.89 / **1.08** / 1.79 / 2.95 GB
≤ 1 / ≤ 1.5 / ≤ 2 GB	51 / 437 / **468 of 500 (94 %)**
> 2 GB	**32, all matplotlib** (versions 3.4–3.7)
Total	≈ 619 GB
Largest repos under the cap	xarray p50 1.91 GB, scikit-learn 1.39 GB

- **On disk ≈ 3× compressed.** Our local django image: 1.1 GB compressed, 4.4 GB on disk (layers 1.5 + 0.76 + 0.67 + 0.16 GB: OS, conda, testbed env, repo).
- **Our wrapper adds 54 MB** (standalone Python + HTTP server), the same on all 20 images we pushed to ECR.
- **Why amd64 still matters:** it is what the m5 lane runs (no cap), and an upper bound for arm64 rebuilds (slide 10).

Source: Docker Hub tags API (`survey_image_sizes.py`); local `docker image inspect`; ECR `describe-images` for `demo_cpu_swebench`; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

SWE-bench is the heavy end: about 2× as many images as Terminal-Bench, with an 8× larger median

Table 8 **Image needs of the agentic-RL workloads we run**

Workload	Images	Size p50 / max	arm64	Fits a microVM
Literature RLVR tasks (`demo_cpu_tasks`)	2	small (stdlib / + scipy)	✓	✓ all
Terminal-Bench	229	128 MB / 6.5 GB	built by us	218 / 229 (7 no arm64, 4 > 2 GB)
SWE-bench Verified	**500**	**1.08 GB** / 2.95 GB	**420** / **500** prebuilt	420 / 500 (all arm64 images ≤ 2 GB)

- Terminal-Bench ships Dockerfiles, so we built arm64 ourselves. SWE-bench has **prebuilt arm64 images** for 420 rows, so most rows need a re-wrap, not a rebuild.
- **Neither the size nor the architecture blocks most rows**; the hard part is the 80 rows with no arm64 image, and arm64 test fidelity.

Source: Terminal-Bench: `demo_cpu_tasks/terminalbench/tasks/images.csv`; SWE-bench: `survey_image_sizes.py`; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

For arm64, reuse the existing builds, rebuild the rest, and keep m5 as the fallback

Table 9 **Options for the arm64-only microVM**

Option	How	Catch	Verdict
Existing arm64 images	Official Docker Hub `sweb.eval.arm64.*` or Epoch AI's GHCR rebuilds, re-wrapped for AgentCore: covers **420** / **500** rows	Tests may behave differently on arm64; Epoch provenance and layout (139 rows Epoch-only)	**Recommended first**
Rebuild for arm64	Build the 80 missing rows' (repo, version) specs with the SWE-bench harness on arm64	Old pinned deps may lack aarch64 wheels	**For the gaps**
Emulate x86 (qemu)	`qemu-x86_64` inside the microVM	binfmt needs capabilities AgentCore drops; far slower tests (not measured)	×
Instance lane (m5)	Stock amd64 images, one m5 per session	≈ 90 s boot, a whole m5 per episode	Fallback (built)

Safety net: the oracle check on every arm64 image. The gold patch must solve the row and the empty patch must not (`swebench_oracle_check.py`); rows that fail go to the Instance lane.

Source: Image registries surveyed by `survey_image_sizes.py` (slide 10); AgentCore capability drop: project finding; m5 boot from the smoke run; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Most rows need nothing for the 2 GB cap; slimming and per-environment images cover the rest

Table 10 **Options for the 2 GB compressed cap**

Option	How	Gain	Catch
Nothing	The cap is compressed (probed), and every existing arm64 image fits (max 1.73 GB)	420 / 500 rows	The 80 rows still to be built; 27 are ≈ 2.9 GB on amd64
Slim	`conda clean -afy`; drop pip/apt caches, docs, `__pycache__`, static libs; flatten layers; trim git history to `base_commit`	Likely enough for matplotlib: its 7 arm64 builds are 1.4–1.73 GB	Re-run the oracle check
Install at boot	Keep the image small; fetch a `conda-pack` tarball from S3 at session start	No size limit	Boot time; needs S3 endpoint or NAT; pre-open hides it
One image per env	Bake the 80 environments; `git checkout base_commit` at session start	500 → **80** images and Runtimes	Instance-specific setup must be replayable

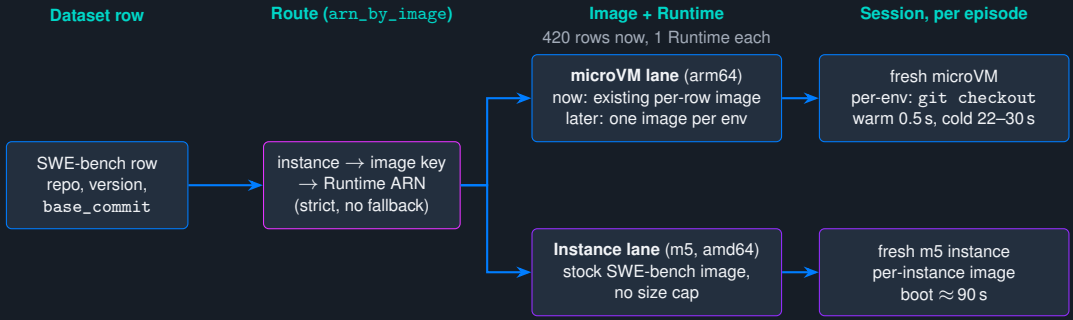
Per-environment images also fix the Runtime quota: one Runtime per row caps us at 1000 rows per account, too few for the full test set (2,294) or SWE-smith (59,136).

Source: Cap probe (slide 8); size survey (slide 12); Runtime quota [Q1]; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Two lanes behind one router: arm64 microVMs for most rows, m5 Instances for the tail

Figure 3 Recommended design: the pool routes each row to a microVM Runtime or an m5 Runtime



- **No new machinery:** the pool already routes strictly by `task.image` → ARN, so a mixed map is all it takes. 420 per-row Runtimes fit the 1000 quota; per-env images matter beyond Verified.
- **Same isolation in both lanes:** one brand-new session per episode; `test_patch` applied only at grading.

Source: microVM open times from the cap probe (slide 9); m5 boot from the smoke run (slide 19); 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Even cold, the microVM lane opens about 3× faster; the Instance lane keeps the exact validated image

Table 11 The two lanes side by side

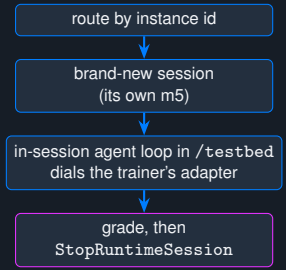
	microVM lane (arm64)	Instance lane (m5)
Image	existing or rebuilt arm64 env image, slimmed if needed	stock SWE-bench image, untouched
Fidelity	new architecture: oracle check required	exactly what SWE-bench validated
Session open	warm **0.5 s** (pre-warm inferred); cold 22–30 s at 1.5–2 GiB (n = 10)	≈ **90 s** pull + boot (p50 91 s)
Cost per episode	one microVM, billed per vCPU-hour and GB-hour used	a whole m5.xlarge
Build effort	re-wrap existing images; rebuild the gaps; validate	per-instance wrap + push
Scale limit	5000 active sessions	instances in the capacity provider

Source: Cap probe and smoke run, both 2026-09-28; session quota [Q2]; AgentCore pricing page; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Demo 1, `demo_cpu_swebench` on AgentCore: the first run used the m5 Instance lane, reusing the Terminal-Bench pool

- **Selection** (`fetch_swebench.py`): 20 "$<$ 15 min fix" SWE-bench tasks, 1–3 F2P, $\leq$ 2 test files, image $\leq$ 3 GB, repos our log parsers support; 16 train / 4 eval.
- **Deploy** (`deploy/deploy_swebench.py`): wrap each stock image with a standalone CPython + the AgentCore HTTP server, push to ECR, one Runtime per task on the `acs_swe_cp` m5 capacity provider.
- **Grading:** reset the test files to `base_commit`, `git apply` the hidden `test_patch`, run the official `test_cmd` in conda env `testbed`, parse with the SWE-bench log parsers.
- **Reused:** `TerminalBenchPool` (strict routing, look-ahead pre-open, heartbeat, retry on a new id).
- **Oracle check:** **1 / 20** so far (`django__django-15127`: gold solves, empty doesn't).



Source: `code/demos/demo_cpu_swebench/` (Runtimes deployed 2026-09-26); oracle results `swebench/oracle.csv`; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

First run on AgentCore: 0 infra errors and the 90 s m5 boot hidden, but 0 of 24 episodes solved

Table 12 Smoke run (RB = 4, NS = 8 intended, Qwen3-4B)

Episodes / infra errors	**24** on 8 SWE-bench tasks, each episode on its own fresh m5 / **0**
Session open	p50 93 s (pull + boot p50 91 s), 0 warm hits
Boot on the critical path	**p50 0** (pre-open); max 143 s on the first batch
Episode time p50	34 s (agent 16 s, grade 1.8 s)
Agent stop	14 / 24 hit the 12-turn budget; 7 ended their turn; 3 hit `max_tokens`
Reward	**0 / 24 solved**, 0 F2P passed; existing (P2P) tests still pass (592 total)
Trainer	CUDA OOM after 2 rollouts (`max-tokens-per-gpu` 24576)

- **The sandbox side works:** the pre-open that hides microVM boot also hides a 90 s m5 boot.
- **Next blockers:** the trainer's memory, then reward sparsity.

In plain English: the machines were ready on time every time; the model just didn't fix any bugs yet.

Source: `code/demos/demo_cpu_swebench/logs/smoke_all_20260928_024714.log`; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Demo 1 now runs on arm64 microVMs: no capacity provider, and a cold open of 22–30 s instead of ≈ 90 s

Table 13 `demo_cpu_swebench`**: first run vs the current version (same 20 rows, harness, grader, recipe)**

	First run: m5 Instance lane	Now: arm64 microVM lane
Image	stock amd64 `sweb.eval.x86_64.*` + server	Epoch AI arm64 build + server, cross-built (buildx + qemu), 0.7–1 GB
Runtime	one per row on capacity provider `acs_swe_cp`	one plain microVM Runtime per row (`acs_swearm_*`), VPC mode, egress none
Per episode	a brand-new session = a whole m5.xlarge	a brand-new session = its own microVM
Session open	p50 93 s (measured, slide 19)	cold 22–30 s at 1.5–2 GiB (cap probe, slide 9)
Idle cost	m5 instances in the capacity provider	nothing beyond the running microVMs
Oracle check	`django-15127`: gold solves, empty doesn't (on m5)	same row passes **locally under qemu** (grade 13.7 s emulated)
Status	deployed 2026-09-26; smoke run done	code switched 2026-09-28; **AWS run pending**

The old pool (20 `acs_swe_*` Runtimes + `acs_swe_cp`) is kept in `swebench/m5_legacy/` until we tear it down; the m5 lane stays the fallback for rows without an arm64 image.

Source: `code/demos/demo_cpu_swebench/` (`README.md`, `swebench/oracle_local.csv`, `m5_legacy/oracle.csv`); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore limits: arm64 and 2 GB leave 80 rows out, one Runtime per row caps the dataset, and idle sessions need a heartbeat

Table 14 **Limitations of the AgentCore demo**

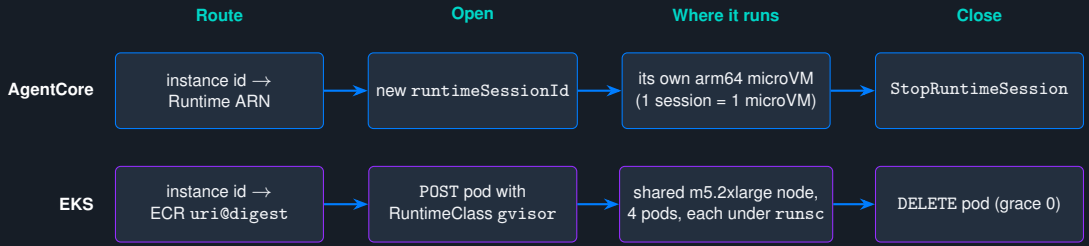
Limitation	Effect	Mitigation
arm64 only [D1]	80 / 500 Verified rows have no arm64 build (all of xarray); arm64 tests may differ from the validated amd64 run	rebuild per env; oracle check per row; m5 lane for the rest
2 GB compressed cap [Q2]	not adjustable; rebuilt images near 2.9 GB won't fit	slim, install at boot, or m5 lane (slide 15)
One Runtime per row [Q1]	1000 Runtimes per account; SWE-smith has 59,136 rows	one image per environment + `git checkout`
Cold open per episode	22–30 s, and every episode is a new session (no warm reuse)	look-ahead pre-open; warm-pool refill under load not measured
Idle reclaim	sessions idle ≈ 15 min are reclaimed [Q1]	`SessionKeeper` heartbeat, always on
Dial-back networking	the in-session loop must reach the trainer's adapter	VPC mode + SG→SG rule on 18100–18163
Not yet shown	arm64 lane never run on AWS; 1 / 20 rows oracle-checked; first run solved 0 / 24 and hit a trainer OOM	AWS oracle check, then smoke (slide 27)

Source: Survey and cap probe (slide 10, slide 8); smoke run (slide 19); quotas [Q1, Q2]; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Demo 2, `demo_cpu_swebench_eks`, swaps only the sandbox: one fresh gVisor pod per episode on a shared EKS node

Figure 4 Same route, agent loop and grader; only how an episode's sandbox is opened and closed changes



- **Unchanged:** the 20 rows, the in-image server on :8080, the in-session agent loop, the hidden-test grader, the pool and the slime recipe. Images are the **stock amd64** ones, shared with the ECS twin (`acs-swe-*` repos).
- **New:** `EksPodSandbox` (drop-in for the AgentCore sandbox); a Kubernetes API client with no `kubectl` (boto3-signed EKS token); a DaemonSet that installs gVisor on each node; a NetworkPolicy (in: :8080 from the trainer; out: only the trainer's adapter ports).
- **Status:** code complete and unit-checked offline; **cluster not created** (render only; `EKS_CREATE=1` builds it).

Source: `code/demos/demo_cpu_swebench_eks/` (`eks_pod_sandbox.py`, `eks_api.py`, `deploy/eks/`); gVisor [G1]; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

EKS limits: nothing measured yet, a syscall tax under gVisor, a shared node, and a cluster to run

Table 15 **Limitations of the EKS + gVisor demo**

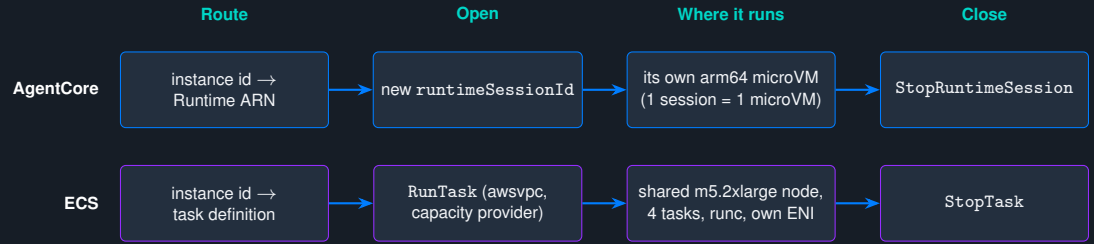
Limitation	Effect	Mitigation
Not deployed	no cluster exists; open time, density and test fidelity are design values, not measurements	`EKS_CREATE=1` on 2 rows first
gVisor syscall tax [G1]	syscall-heavy suites (django) run slower; some tests may break under `runsc`	oracle check on gVisor pods; `EKS_RUNTIME_CLASS=` for a runc A/B
Weaker boundary	a user-space kernel per pod, but pods share the node's CPU, memory bandwidth, disk and page cache	one pod per episode, never reused; no service-account token
gVisor install	a privileged DaemonSet edits containerd's config; the AMI's containerd layout is a risk	RuntimeClass only schedules on nodes labelled ready
Image disk	3–6 GB per image per node; kubelet evicts images above 85 % disk	pre-pull DaemonSet; 300 GiB gp3 per node
Capacity	4 nodes × 4 pods = 16 concurrent vs 64 pre-opened	raise `EKS_NODES` or lower the pre-open
You operate	control plane (billed), node group, autoscaler, NetworkPolicy, upgrades; `eksctl` on the host	render first; idempotent up / down

Source: `code/demos/demo_cpu_swebench_eks/README.md` (design, known risks); VPC CNI network policy [E1]; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Demo 3, `demo_cpu_swebench_ecs`: one fresh ECS task with its own ENI per episode, on a shared EC2 node

Figure 5 Same route, agent loop and grader; the task gets its own ENI and security group



- **Unchanged:** rows, server, agent loop, grader, pool and recipe; the amd64 images are the EKS twin's (one task definition per image, pinned by digest).
- **New:** `EcsTaskSandbox` (drop-in); one poller thread with batched `DescribeTasks`; `ecs_deploy.py` (pure boto3: ASG + capacity provider with managed scaling, ENI trunking). Hardening: all capabilities dropped, no task role, IMDS blocked; the task SG lets it dial back only the adapter ports.
- **Status:** code complete, tested offline against fake AWS clients; **never deployed** (render only; `ECS_CREATE=1` builds it).

Source: `code/demos/demo_cpu_swebench_ecs/` (`ecs_task_sandbox.py`, `deploy/ecs_deploy.py`); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

ECS limits: the node kernel is shared, so this is only for trusted-model RL, and it is not deployed yet

Table 16 **Limitations of the ECS on EC2 demo**

Limitation	Effect	Mitigation
Shared kernel	a plain runc container: a kernel exploit escapes to the node and its other episodes	caps dropped, no task role, IMDS blocked; add gVisor/-Firecracker yourself
Not deployed	nothing measured; the trainer's role has no `ecs:*` permission today	grant ECS/EC2/ASG/IAM rights; `ECS_CREATE=1` on 2 rows
No per-task disk quota	tasks share the node's Docker storage; a runaway episode can fill it	size `ECS_NODE_DISK_GIB` with headroom
ENI trunking	must be on before nodes register, or a node gets only 3 task ENIs	`up` sets it first; recycle older nodes
RunTask rate limits	per account and region; bursts get throttled	adaptive retries; pre-open cap 64
No idle reclaim	a leaked task runs (and bills) until killed	entrypoint `timeout` 7200 s; `down` stops all ours
Capacity	4 nodes $\times$ 4 tasks = 16 concurrent vs 64 pre-opened	raise `ECS_NODES_MAX` or lower the pre-open

Source: `code/demos/demo_cpu_swebench_ecs/README.md` (design, limitations); checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore isolates best with the least to run; EKS and ECS run every image

Table 17 The three SWE-bench demos side by side

	AgentCore (arm64 microVM)	EKS + gVisor	ECS on EC2
Isolation	**own microVM** per episode [D1]	gVisor kernel per pod; node shared [G1]	container only; **node kernel shared**
Image	arm64, $\leq$ 2 GB compressed	stock amd64, any size	stock amd64, any size
Verified rows	420 / 500 (rest: m5 lane)	500 / 500	500 / 500
Per episode	1 microVM	1.5 vCPU / 6 GiB pod, 4 per node	1.5 vCPU / 8 GiB task, 4 per node
Open	cold **22–30 s** (measured)	pod + `runsc`; not measured	`RunTask` + ENI; not measured
Exec path	AWS data plane (Invoke API)	HTTP to the pod IP, gVisor syscall tax	HTTP to the task IP, runc (fastest)
Idle / leaks	reclaimed at 15 min: heartbeat needed	no reclaim; heartbeat = liveness; 2 h deadline	no reclaim; heartbeat = liveness; 2 h timeout
Egress / creds	no-egress subnets + SG; no credentials	NetworkPolicy: adapter ports only; no SA token	task SG: adapter ports only; no role, no IMDS
You operate	Runtimes (one per row); no idle cost	cluster, nodes, gVisor, policy, upgrades; control plane billed	ASG, capacity provider, task defs; nodes billed
Status	m5 run done; arm64: local oracle pass	code complete, not deployed	code complete, not deployed

In plain English: AgentCore: own VM per episode, no fleet, but arm64 $\leq$ 2 GB only. EKS / ECS: any image, denser, weaker walls, a fleet to run.

Source: slide 20–slide 25; demo READMEs under `code/demos/demo_cpu_swebench*`; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Next: run the arm64 lane on AWS, then fix the trainer OOM; EKS and ECS only when we decide to create them

Table 18 **Next steps**

Next	Why
Push the 20 arm64 images, register the microVM Runtimes, oracle-check all 20 on AWS	Only usable rows should reach training; validate arm64 fidelity
Smoke run on the microVM lane; lower `MAX_TOKENS_PER_GPU` for the OOM	Measure open time on real rows; get past step 2
Tear down the m5 pool (`m5_legacy`) once the arm64 lane works	Stop paying for idle m5 capacity
Measure warm-pool refill under sustained load	$\approx$ 10 pre-warmed seen only on a fresh Runtime
Per-environment images + checkout at session start	500 $\rightarrow$ 80 Runtimes; scale beyond Verified
Easier instances, more turns, partial credit	0 / 24 solves means no GRPO gradient
EKS / ECS: create on 2 rows, oracle check, smoke (ECS needs `ecs:*` rights first)	Fill the “not measured” cells of slide 26

Source: Smoke run and cap probe, 2026-09-28; demo READMEs.

© 2026, Amazon Web Services, Inc. or its affiliates.

Appendix

Per-repo arm64 coverage · references

Per repo, arm64 coverage is complete for 7 of 12 repos and zero only for xarray

Table 19 SWE-bench Verified image coverage and median compressed size, by repo

Repo	Rows	Official arm64	Epoch arm64	No arm64	arm64 p50	amd64 p50
django	231	141	229	2	941 MB	1,137 MB
sympy	75	67	75	0	857 MB	1,032 MB
sphinx	44	34	44	0	787 MB	1,048 MB
matplotlib	34	2	7	**27**	1,767 MB	2,992 MB
scikit-learn	32	4	7	**25**	1,006 MB	1,426 MB
astropy	22	7	18	4	856 MB	1,084 MB
xarray	22	0	0	**22**	—	1,953 MB
pytest	19	10	19	0	689 MB	979 MB
pylint	10	7	10	0	707 MB	1,033 MB
requests	8	7	8	0	672 MB	968 MB
seaborn	2	1	2	0	812 MB	1,233 MB
flask	1	1	1	0	680 MB	1,093 MB
Total	**500**	**281**	**420**	**80**	916 MB	1,104 MB

arm64 p50 = the smaller of the two arm64 images per row. Epoch's amd64 rebuilds also exist for all 500 rows (473 $\leq$ 2 GB). Back to slide 10.

Source: `swebench/verified_image_sizes.csv` (`survey_image_sizes.py`); MB = 2^{20} bytes, compressed; checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

References

Ref	Source	Supports
[Q1]	AgentCore quotas: `https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/bedrock-agentcore-limits.html`	1000 Runtimes / account; 15-min request and idle timeouts
[Q2]	`aws service-quotas get-service-quota -service-code bedrock-agentcore` (us-west-2, 2026-09-28)	Image 2048 MB, not adjustable (L-0A9E32B3); 5000 active sessions; 25 new / s
[D1]	`https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html`	Session = dedicated microVM; image must be linux/arm64
[D2]	Project measurement, `demo_cpu_swebench` (`acs_swe_cp`, m5.xlarge)	amd64 Instances, 1 session = 1 instance, $\approx$ 90 s boot
[P1]	Project probe, `probe_image_cap.py` (us-west-2, 2026-09-28)	Cap = 2048 MiB compressed, enforced at create; cold / warm open times
[G1]	gVisor: `https://gvisor.dev/docs/`; Kubernetes RuntimeClass: `https://kubernetes.io/docs/concepts/containers/runtime-class/`	User-space kernel per sandbox; `runsc` handler per pod
[E1]	`https://docs.aws.amazon.com/eks/latest/userguide/cni-network-policy.html`	VPC CNI network policies on EKS
[S1]	`princeton-nlp/SWE-bench_Verified`; Docker Hub `swebench/sweb.eval.{x86_64,arm64}.*`	500 instances, 12 repos, 80 envs; sizes, architectures
[S2]	GHCR `epoch-research/ swe-bench.eval.{arm64,x86_64}.*` (Epoch AI)	Community rebuilds, arm64 sizes
[S3]	Hugging Face `SWE-Gym/SWE-Gym`, `SWE-bench/SWE-smith`	2,438 and 59,136 training rows

Source: All links and quota values checked 2026-09-28.

© 2026, Amazon Web Services, Inc. or its affiliates.

Thank you

Kaige Yang · Senior Applied Scientist, AWS